实验二 加法器的设计及应用

一、实验目的

1.掌握 Verilog 语言框架，编程及调试的方法。
2.熟悉 Verilog 的基本语法。
3.掌握 Vivado 开发平台及 FPGA 开发板的使用。

二、实验内容

1.完成 1 位半加器、1 位全加器模块的实现与测试；
2.利用 1 位全加器实现 2 位全加器，并烧录到开发板进行验证；
3.实现并测试 16/32 位全加器，记录、分析设计代码、仿真波形和 RTL 电路图等。
4.用 Verilog 的 generate 语句实现（选做）。

三、实验要求

1.说明电路功能，分析设计、仿真代码和电路图。
2.分析仿真波形，观察输入输出是否与预期电路功能相符（测试要全面，关注特殊情况的测试）。
3.记录设计和调试过程。

四、实验过程及结果分析

对电路功能、设计代码、电路图、测试结果等进行简要分析说明，不能只贴截图。

4.1 1 位半加器和全加器模块的设计与测试

1、半加器：

```
module halfadder(S, C, A, B);
    input A, B;
    output S, C;
    xor (S, A, B);
    and (C, A, B);
endmodule
```

分析：S 为输出，由 A 异或 B 得到；C 为进位，由 A 与 B 得到。

2、全加器：

```
module fulladder(S, Co, A, B, Ci);
    input A, B, Ci;
    output S, Co;
    wire S1, D1, D2;
    halfadder HA1(S1, D1, A, B);
    halfadder HA2(.S(S), .C(D2), .A(S1), .B(Ci));
    or G1(Co, D2, D1); //先输出，后输入
endmodule
```

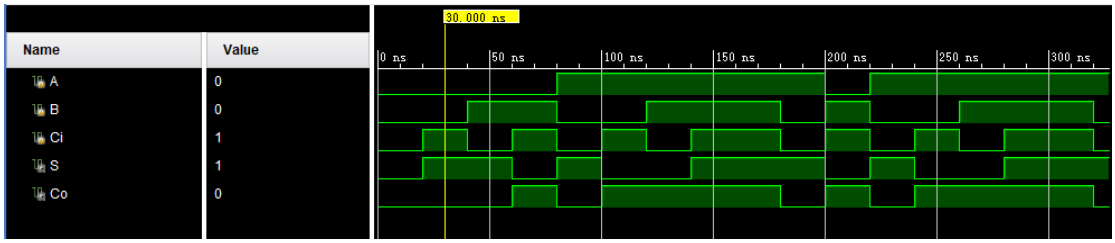
分析：全加器加入进位输入，可以调用两个半加器实现。先将 A、B 相加的结果和进位存入 S1 和 D1，然后将 S1 与进位输入 Ci 相加，得到最终输出结果 S 与另一个进位 D2。计算 D2 或 D1，得到最终的进位输出 Co。

3、测试文件：

```
module sim_addr;
    reg A, B;
    reg Ci;
    wire S;
    wire Co;
    initial begin
        A = 1'b0; B = 1'b0; Ci = 0;
    end
    always begin
        #20 Ci = 1'b1;
        #20 B = B + 1'b1; Ci = ~Ci;
        #20 {A, B, Ci} = 3'b011;
        #20 {A, B, Ci} = 3'b100;
        repeat(3) begin
            #20 {A, B, Ci} = {A, B, Ci} + 1'b001;
        end
    end
    fulladder T(S, Co, A, B, Ci);
endmodule
```

分析：测试从{A, B, Ci} = {0, 0, 1}递增至{1, 1, 1}的全部情况。

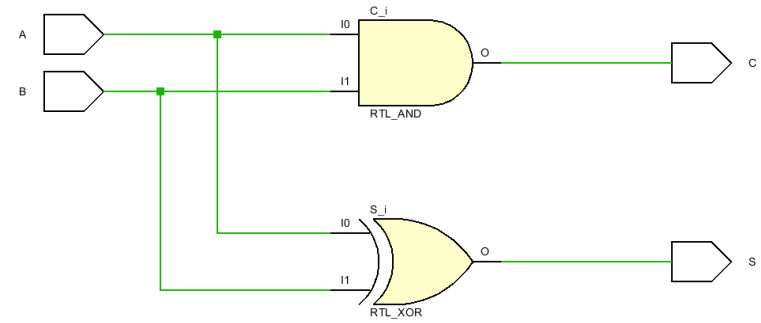
4、SIMULATION：



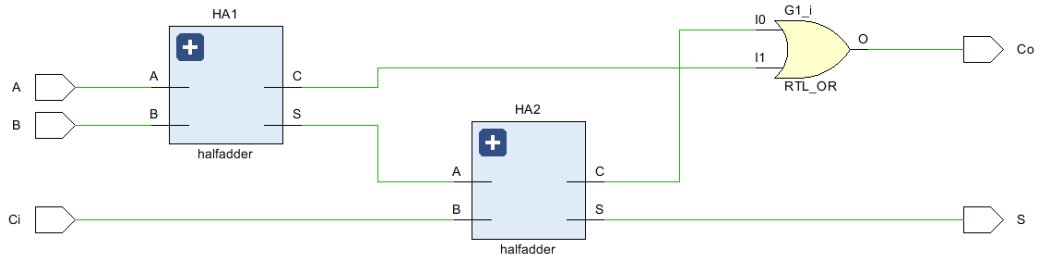
分析：如图，以 A=B=Ci=1 为例分析，S 和 Co 都为 1，符合预期，说明功能正确。

5、RTL ANALYSIS:

半加器：



全加器：



4.2　2 位全加器模块的设计及 FPGA 验证

1、2 位全加器代码：

```
module addr_2bit(S, Co, A, B, Ci);
    input [1:0] A, B;
    input Ci;
    output [1:0] S;
    output Co;
    wire C;

    fulladder FA0(S[0], C, A[0], B[0], Ci),
              FA1(S[1], Co, A[1], B[1], C);
endmodule
```

分析：2 位全加器可通过调用两个全加器，对每位分别计算来实现。先以 Ci 为进位输入，计算 A0+B0，将进位输出传入 C，输出传入 S0；再以 C 为进位输入，计算 A1+B1，输出 S1，进位即为结果 Co

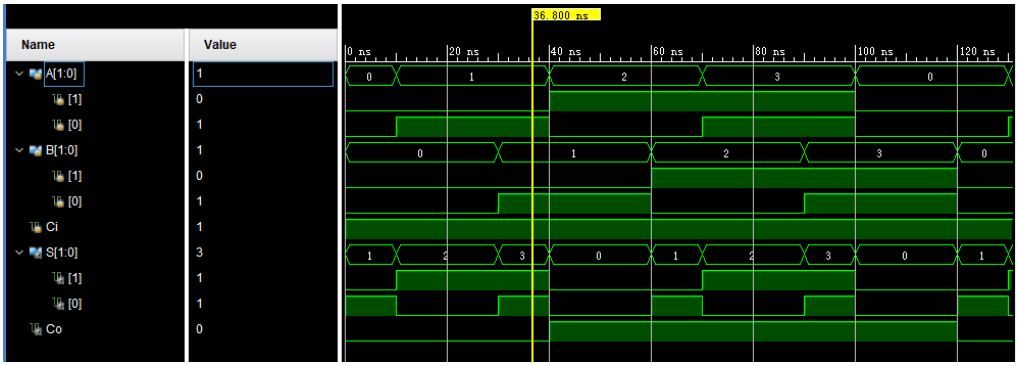
2、测试文件：

```
module sim_addr_2bit;
    reg [1:0] A, B;
    reg Ci;
    wire [1:0] S;
    wire Co;
```

```
    initial begin
        A = 2'b00; B = 2'b00; Ci = 1;
    end
    always begin
        #10 A = A + 1'b1;
        #20 B = B + 1'b1;
    end
    addr_2bit T(S, Co, A, B, Ci);
endmodule
```

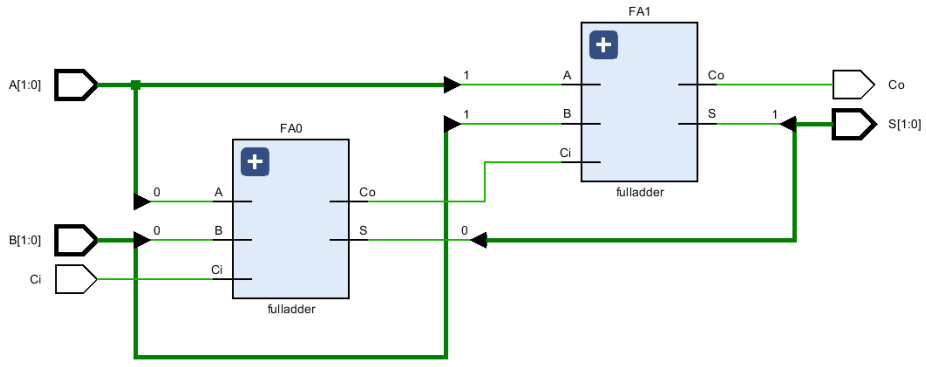
分析：将 A 与 B 按不同时间间隔+1，由此可观察多种不同情况的输出结果。

3、SIMULATION:



分析：以 A=01，B=01，Ci=1 为例，先计算 A0+B0+Ci=1+1+1，S0=1，C=1；再计算 A1+B1+C，得到 S1=1，所以得到最终结果 S=11，Co=0，符合预期功能。

3、RTL ANALYSIS：



4、通过 7 位二进制数设置数码管：

```
module hex7seg(
    input wire [3:0] S,
    output wire [3:0] an,
    output reg [6:0] seg);
    assign an = 4'b1110;

    always @(*) case(S)
        0: seg = 7'b1000000;
```

```
        1: seg = 7'b1111001;
        2: seg = 7'b0100100;
        3: seg = 7'b0110000;
        4: seg = 7'b0011001;
        5: seg = 7'b0010010;
        6: seg = 7'b0000010;
        7: seg = 7'b1111000;
        8: seg = 7'b0000000;
        9: seg = 7'b0010000;
        'hA: seg = 7'b0001000;
        'hB: seg = 7'b0000011;
        'hC: seg = 7'b1000110;
        'hD: seg = 7'b0100001;
        'hE: seg = 7'b0000110;
        'hF: seg = 7'b0001110;
        default: seg = 7'b0000001;
    endcase
endmodule
```

分析：用 7 位二进制数代表数码管的显示灯，低有效。通过不同序列构成 0-F 的输出。

5、配置好数码管显示的 2 位全加器：

```
module addr2b_hex7seg(
    input wire Ci,
    input wire [1:0] A, B,
    output wire [1:0] S,
    output wire Co,
    output wire [3:0] an,
    output wire [6:0] seg);

    addr_2bit addr(S, Co, A, B, Ci);
    hex7seg hb({'b0, Co, S}, an, seg);
endmodule
```

分析：加入 an 和 seg，烧录时实现数码管显示数字功能。

6、约束文件:

```
## Switches
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {B[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]
set_property PACKAGE_PIN W17 [get_ports {B[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]
```

```
## LEDs
set_property PACKAGE_PIN U16 [get_ports {S[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {S[0]}]
set_property PACKAGE_PIN E19 [get_ports {S[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {S[1]}]
set_property PACKAGE_PIN U19 [get_ports Co]
    set_property IOSTANDARD LVCMOS33 [get_ports Co]


##7 segment display
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

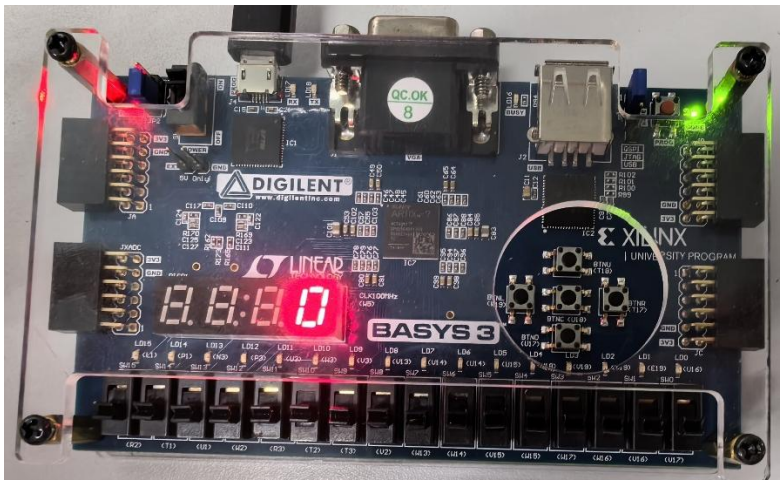

##Buttons
set_property PACKAGE_PIN U18 [get_ports Ci]
    set_property IOSTANDARD LVCMOS33 [get_ports Ci]
```

分析：注意 Ci 为 U18.

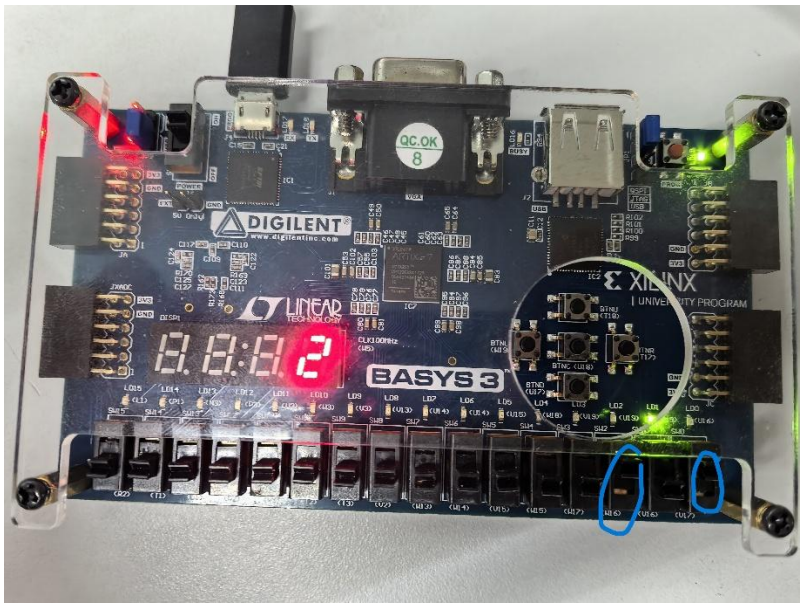
7、烧录结果展示与分析：

三个 LED，分别为 Co、S1、S0；中间按钮为 Ci；右下侧四个按键为 B1、B0、A1、A0

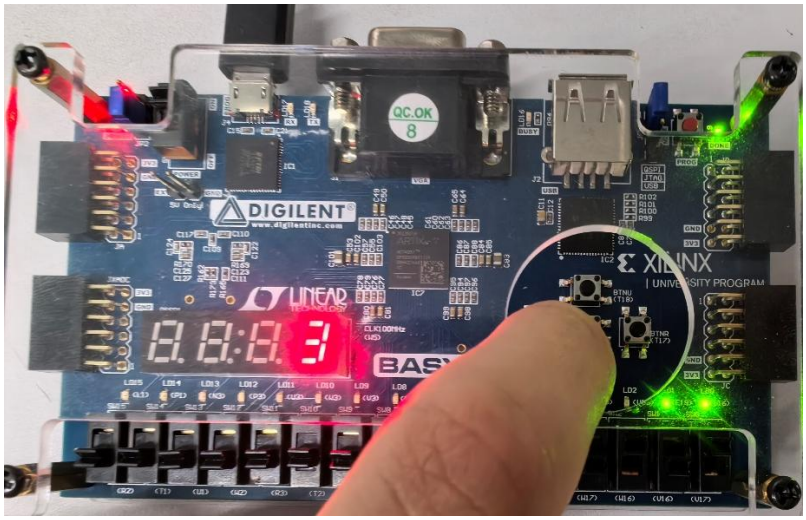
初始状态：



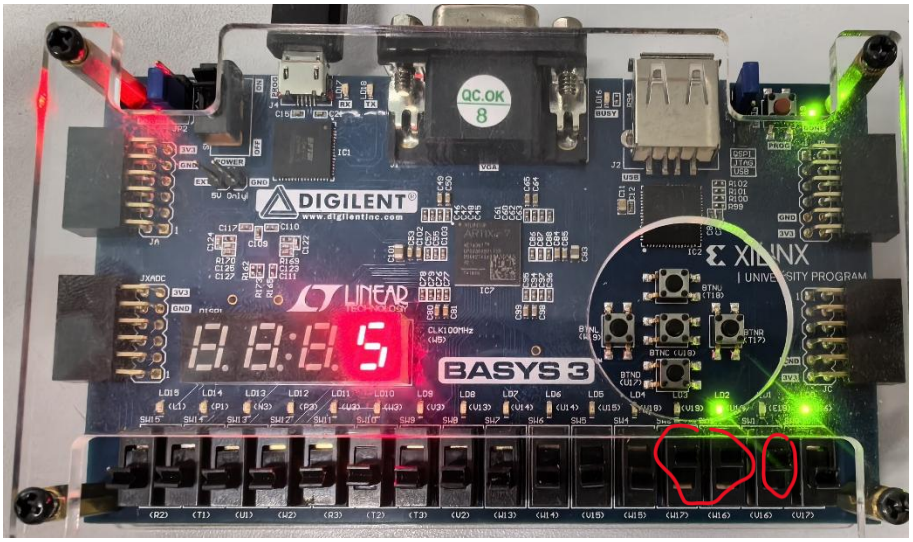
输入为 01 和 01，输出 2，第二个 LED 亮（代表结果为 010）



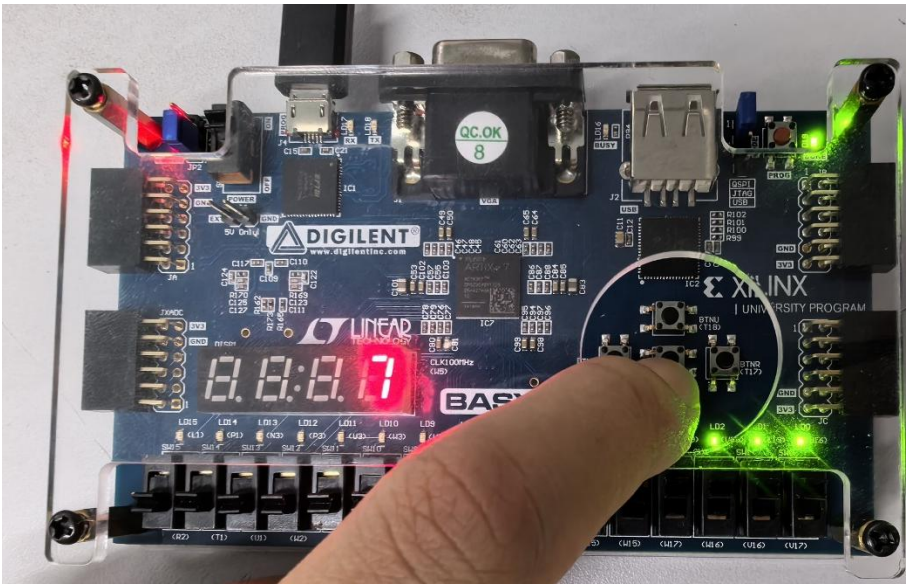
在上面情况下加入 Ci，输出 3，第一、二个 LED 亮（结果为 011）



输入 11 和 10，输出 5，第一、三个 LED 亮（代表 101）：



输入 11、11 和 Ci，输出 7，LED 全亮（代表 111），这是该 2 位加法器的最大结果



4.3　16/32 位全加器的设计与测试

1、4 位全加器：

```
module addr_4bit(
    input [3:0] A, B,
    input C1,
    output [3:0] S,
    output Ch);
    wire C0, C1, C2;

    fulladder FA0(S[0], C0, A[0], B[0], C1),
              FA1(S[1], C1, A[1], B[1], C0),
              FA2(S[2], C2, A[2], B[2], C1),
```

```
            FA3(S[3], Ch, A[3], B[3], C2);
endmodule
```

分析：通过 4 个全加器级联实现，每级的 Co 接下一级的 Ci，其他输入输出按位进行计算。

2、16 位全加器：

```
module addr_16bit(
    input [15:0] A, B,
    input Cin,
    output [15:0] S,
    output Cout);
    wire C3, C7, C11;

    addr_4bit FA0 (.A(A[3:0]), .B(B[3:0]), .Cl(Cin), .S(S[3:0]), .Ch(C3)),
              FA1 (.A(A[7:4]), .B(B[7:4]), .Cl(C3), .S(S[7:4]), .Ch(C7)),
              FA2 (.A(A[11:8]), .B(B[11:8]), .Cl(C7), .S(S[11:8]), .Ch(C11)),
              FA3
(.A(A[15:12]), .B(B[15:12]), .Cl(C11), .S(S[15:12]), .Ch(Cout));
endmodule
```

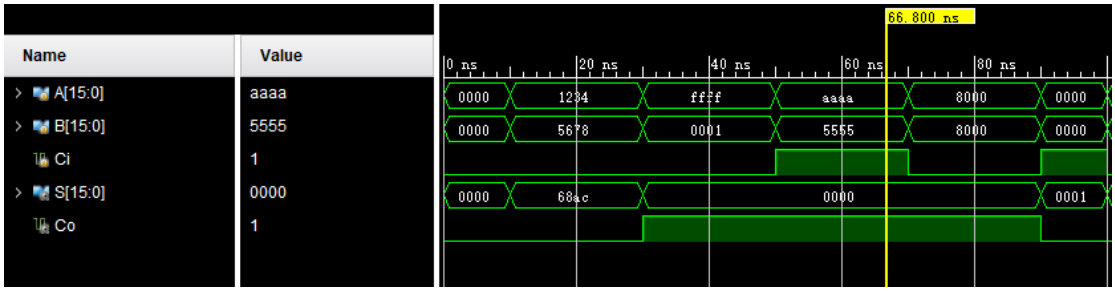
分析：通过 4 个 4 位全加器实现，以 4 位为一个区间传入各级，原理与 4 位全加器近似。

3、仿真文件：

```
module sim_addr_16bit;
    reg [15:0] A, B;
    reg Ci;
    wire [15:0] S;
    wire Co;
    initial begin
        A = 16'h0000; B = 16'h0000; Ci = 0;
    end
    always begin
        #10 A = 16'h1234; B = 16'h5678; Ci = 0;  // 无进位加法
        #20 A = 16'hFFFF; B = 16'h0001; Ci = 0;  // 进位传播测试
        #20 A = 16'hAAAA; B = 16'h5555; Ci = 1;  // 带进位输入的全 1 模式
        #20 A = 16'h8000; B = 16'h8000; Ci = 0;  // 溢出测试
        #20 A = 16'h0000; B = 16'h0000; Ci = 1;  // 仅进位测试
    end
    addr_16bit T(A, B, Ci, S, Co);
endmodule
```

分析：因为 16 位情况过多，所以在此处构造了五种可能的情况进行仿真。

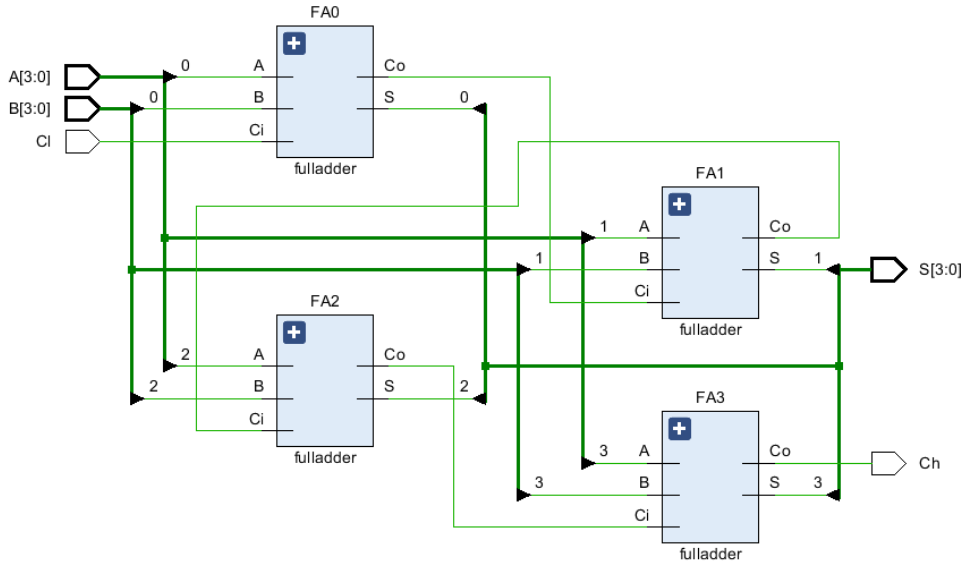
4、SIMULATION：



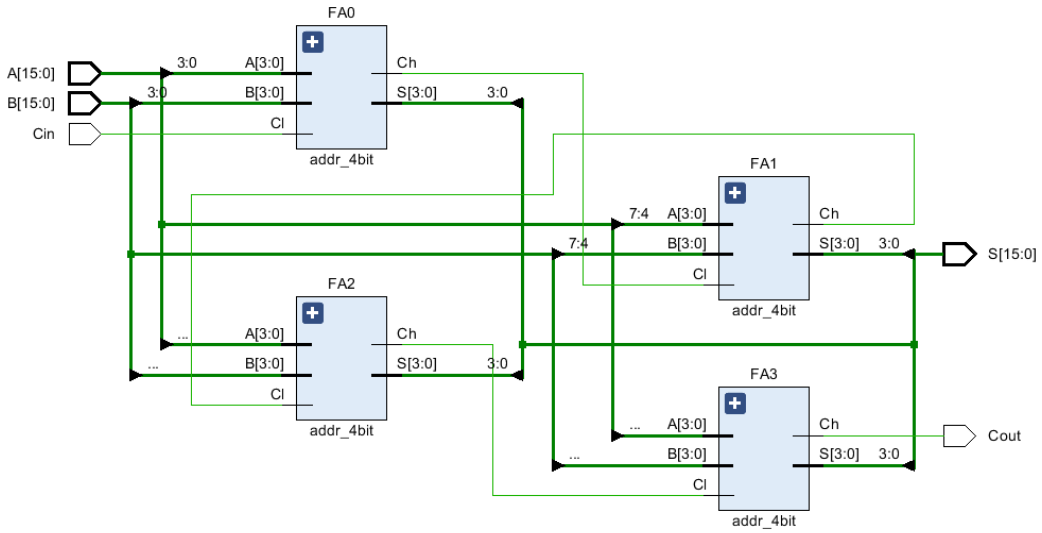
分析：1234+5678=68ac，可验证加法器基础功能正确；ffff+0001=0000，进位，可验证进位功能正确；aaaa+5555=ffff，加入 Ci 后 S=0000，有进位输出 Co；其余两种情况经过验证后，均正确，即可验证该 16 位全加器的输出与进位功能正常实现。

5、RTL ANALYSIS：

4 位全加器：



16 位全加器：



4.4 generate块实现16/32位全加器的设计与测试(选做)

要求：用 Verilog 的 generate 块实现。

1、16 位全加器：

```
module addr_16bits(
    input [15:0] A,
    input [15:0] B,
    input Cin,
    output [15:0] S,
    output Cout);
    wire [3:0] carry;  // 存储进位输出
    assign carry[0] = Cin;  // 最低位进位输入

    genvar i;
    generate
        for (i = 0; i < 4; i = i + 1) begin : adder_block
            addr_4bit FA (
                .A(A[(i*4 + 3):(i*4)]),  // 每 4 位一组输入，思路同上
                .B(B[(i*4 + 3):(i*4)]),
                .Cl(carry[i]),
                .S(S[(i*4 + 3):(i*4)]),   // 每 4 位一组结果
                .Ch(carry[i + 1])
            );
        end
    endgenerate

    assign Cout = carry[3];  // 最终进位输出
endmodule
```

分析：通过使用 generate 块，可以通过四重循环来实例化 4 个 4 位全加器，不用一个一个列举。

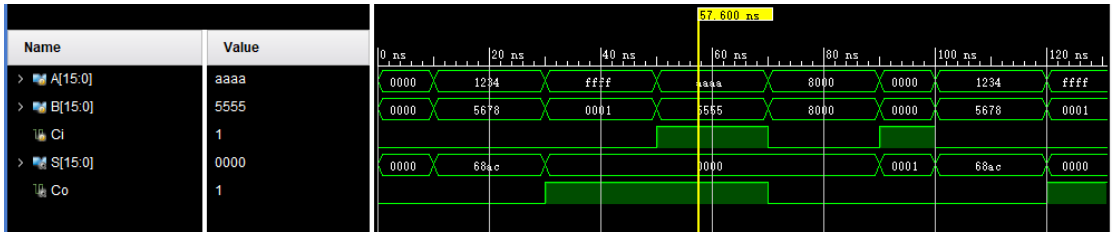
2、仿真文件：

```
module sim_addr_16bits;
    reg [15:0] A, B;
    reg Ci;
    wire [15:0] S;
    wire Co;
    initial begin
        A = 16'h0000; B = 16'h0000; Ci = 0;
    end
    always begin
        #10 A = 16'h1234; B = 16'h5678; Ci = 0;  // 无进位加法
```

```
        #20 A = 16'hFFFF; B = 16'h0001; Ci = 0;  // 进位传播测试
        #20 A = 16'hAAAA; B = 16'h5555; Ci = 1;  // 带进位输入的全 1 模式
        #20 A = 16'h8000; B = 16'h8000; Ci = 0;  // 溢出测试
        #20 A = 16'h0000; B = 16'h0000; Ci = 1;  // 仅进位测试
    end
    addr_16bits T(A, B, Ci, S, Co);
endmodule
```

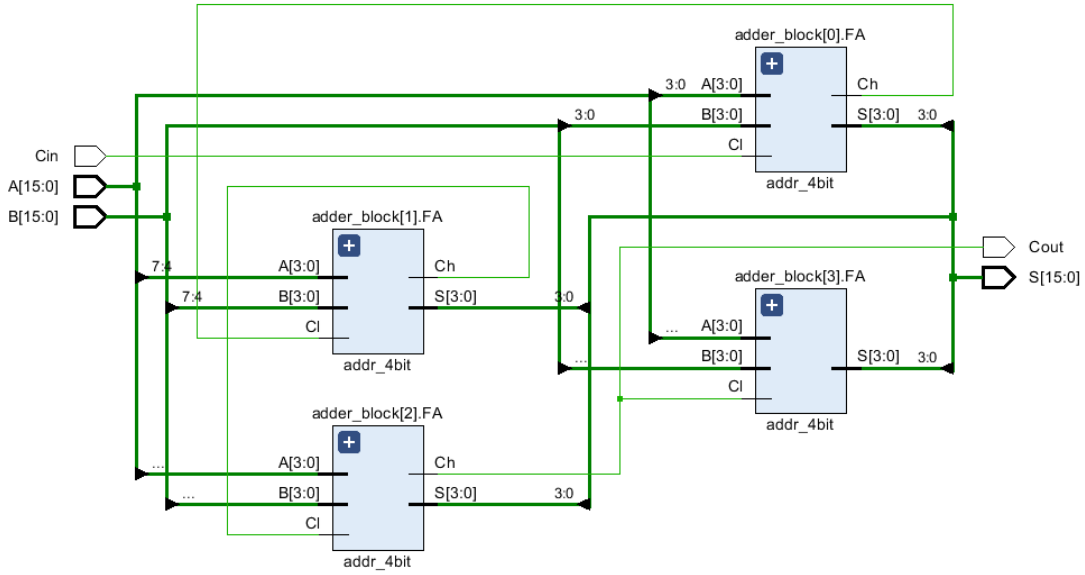
分析：同上

3、SIMULATION：



分析：同上

4、RTL ANALYSIS：



五、调试和心得体会

记录实验过程中遇到的问题和调试解决过程，心得体会等。

1、问题调试：

（1） assign Cout = carry[3]：写成 carry[4]，在仿真时发现 Co 无法正常输出，遂改正。应注意 Verilog 语言数组的标号。

（2） 16 位全加器输入情况太多，无法全部仿真：挑选几个有代表性的情况进行仿真，即可验证电路功能。

2、心得体会：

通过本次加法器的设计与应用实验，我对 Verilog 语言的模块化编程思想和层次化设计方法有了更深刻的理解。在编写半加器、全加器以及多位级联加法器的过程中，我体会到了模块复用和端口映射的重要性，也熟悉了 generate 语句对批量实例化电路的优势；在仿真环节，通过对多种测试情况的验证，我体会到完整测试覆盖对保证电路功能正确性的必要性。对 BASYS 的实际烧录让我切身体会到从代码到硬件实现的全过程，在编辑约束文件的过程中也加深了我对硬件引脚约束的认识。

这次实验过后，我对 vivado 工具与 verilog 语言有了更强的理解与掌握，对加法器与半加器的熟悉度大大提高，能够有效地重新利用到我对数电的学习当中。